Experiment No: 6 (Software) **Date: 02.10.2022**

Multiplexer and Demultiplexer

Aim:

To implement 4:1 mux and 1:4 demux using logic gates in verilog programming.

Software Required: ModelSim

Procedure:

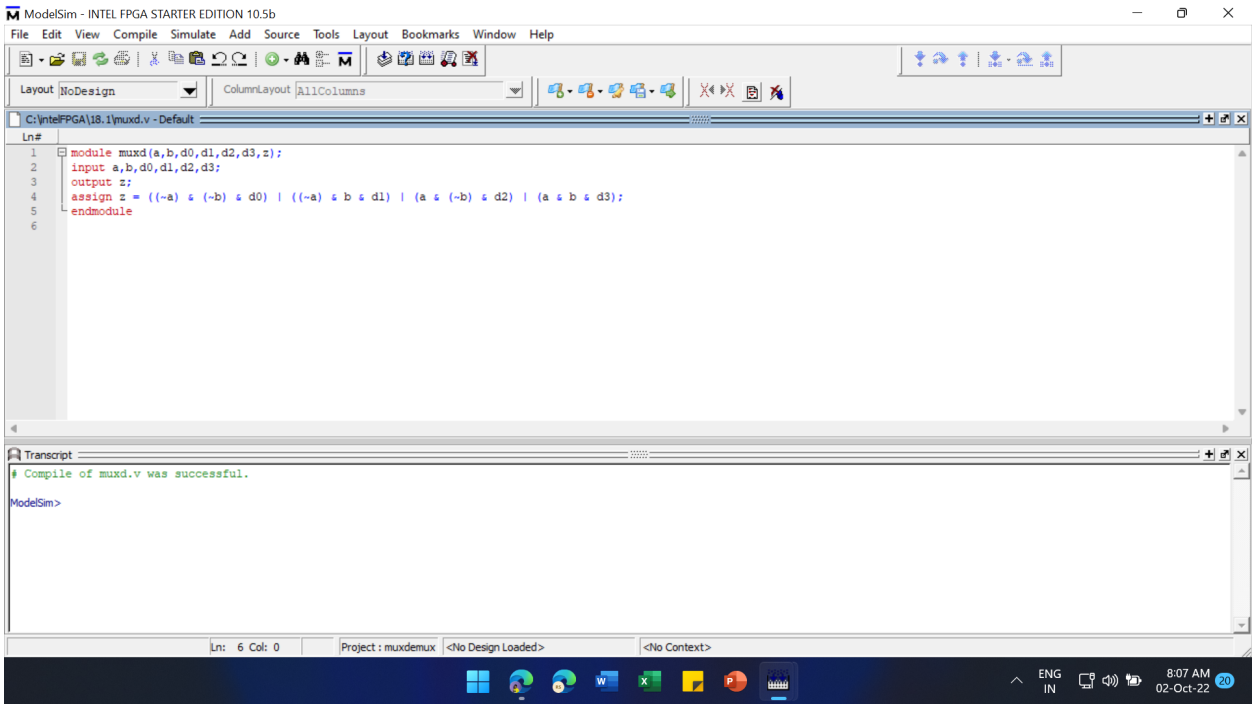
1. Open ModelSim.
2. Create New Project, then create a new file by selecting the verilog in the drop-down.
3. Type the code, then compile it.
4. Type the test bench, compile and simulate it.
5. Click add wave in the window and then run it.

Verilog Code:

Multiplexer:

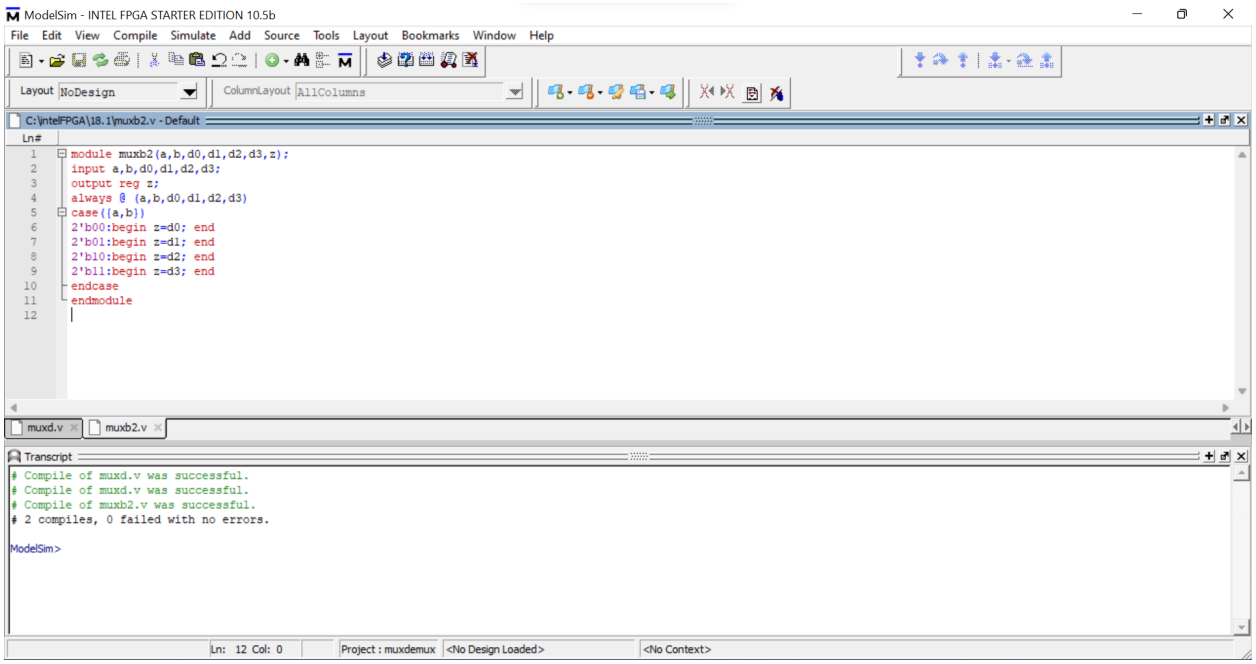
<u>Data-Flow Modeling:</u>

```
module muxd(a,b,d0,d1,d2,d3,z);
input a,b,d0,d1,d2,d3;
output z;
assign z = ((~a) & (~b) & d0) | ((~a) & b & d1) | (a & (~b) & d2) | (a & b & d3);
endmodule
```



<u>Behavioral Modeling:</u>

```
module muxb2(a,b,d0,d1,d2,d3,z);
input a,b,d0,d1,d2,d3;
output reg z;
always @ (a,b,d0,d1,d2,d3)
case({a,b})
2'b00:begin z=d0; end
2'b01:begin z=d1; end
2'b10:begin z=d2; end
2'b11:begin z=d3; end
endcase
endmodule
```

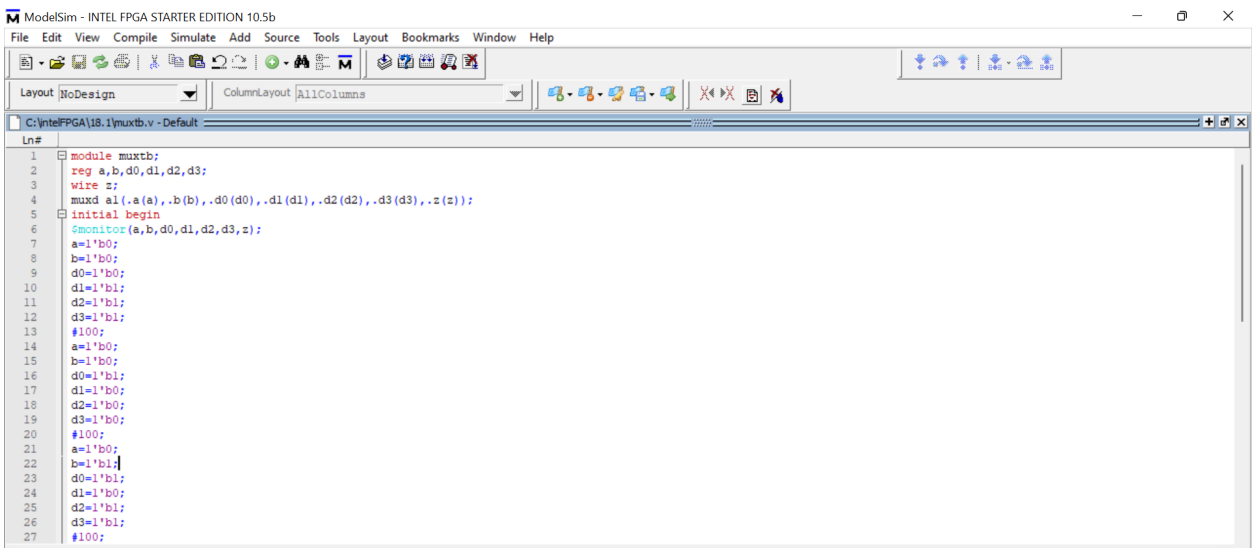


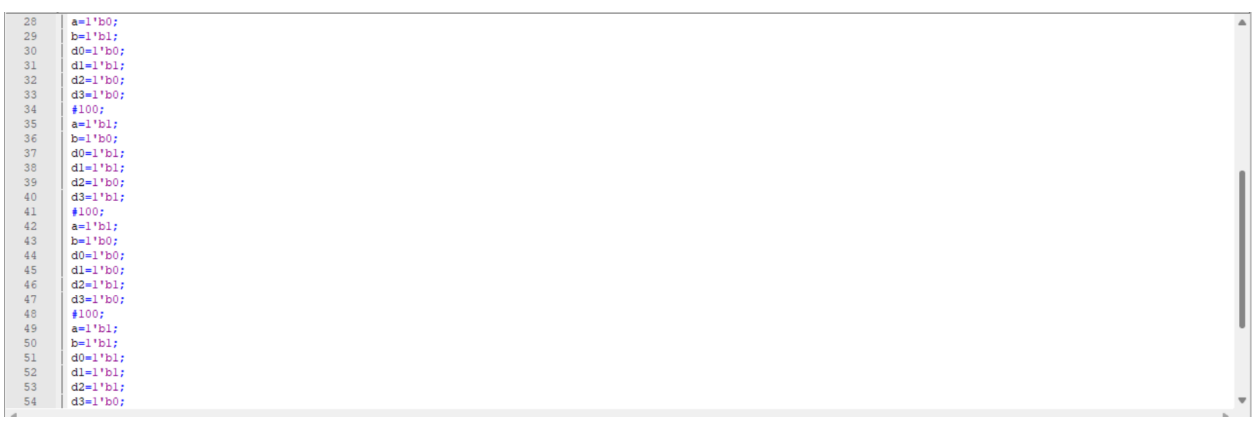
<u>Test Bench:</u>

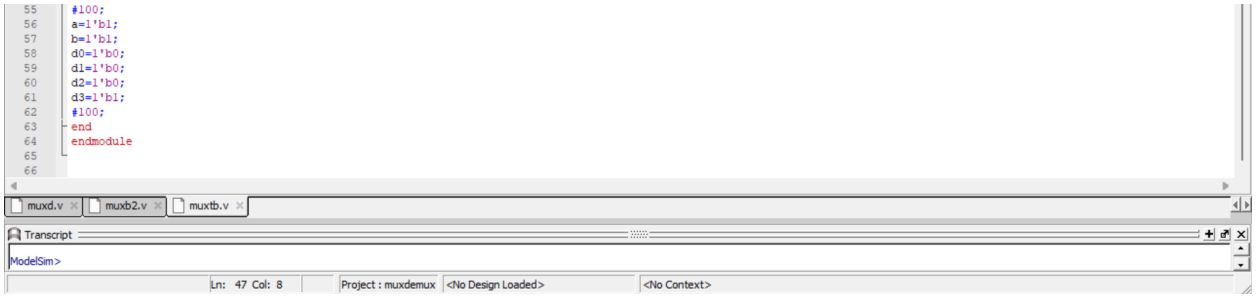
```
module muxtb;
reg a,b,d0,d1,d2,d3;
wire z;
muxd a1(.a(a),.b(b),.d0(d0),.d1(d1),.d2(d2),.d3(d3),.z(z));
initial begin
$monitor(a,b,d0,d1,d2,d3,z);
a=1'b0;
b=1'b0;
d0=1'b0;
d1=1'b1;
d2=1'b1;
d3=1'b1;
#100;
a=1'b0;
b=1'b0;
d0=1'b1;
d1=1'b0;
d2=1'b0;
d3=1'b0;
```

```
#100;
a=1'b0;
b=1'b1;
d0=1'b1;
d1=1'b0;
d2=1'b1;
d3=1'b1;
#100;
a=1'b0;
b=1'b1;
d0=1'b0;
d1=1'b1;
d2=1'b0;
d3=1'b0;
#100;
a=1'b1;
b=1'b0;
d0=1'b1;
d1=1'b1;
d2=1'b0;
d3=1'b1;
#100;
a=1'b1;
b=1'b0;
d0=1'b0;
d1=1'b0;
d2=1'b1;
d3=1'b0;
#100;
a=1'b1;
b=1'b1;
d0=1'b1;
d1=1'b1;
d2=1'b1;
```

```
d3=1'b0;
#100;
a=1'b1;
b=1'b1;
d0=1'b0;
d1=1'b0;
d2=1'b0;
d3=1'b1;
#100;
end
endmodule
```



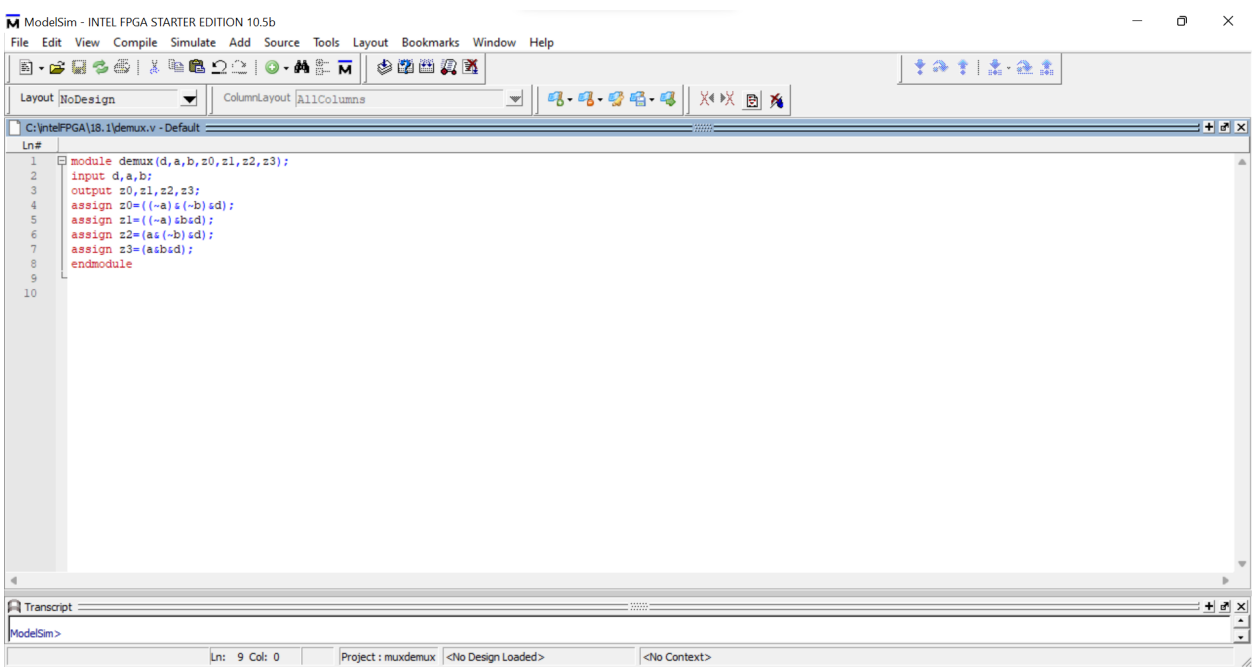




Demultiplexer:

<u>Data-Flow Modeling:</u>

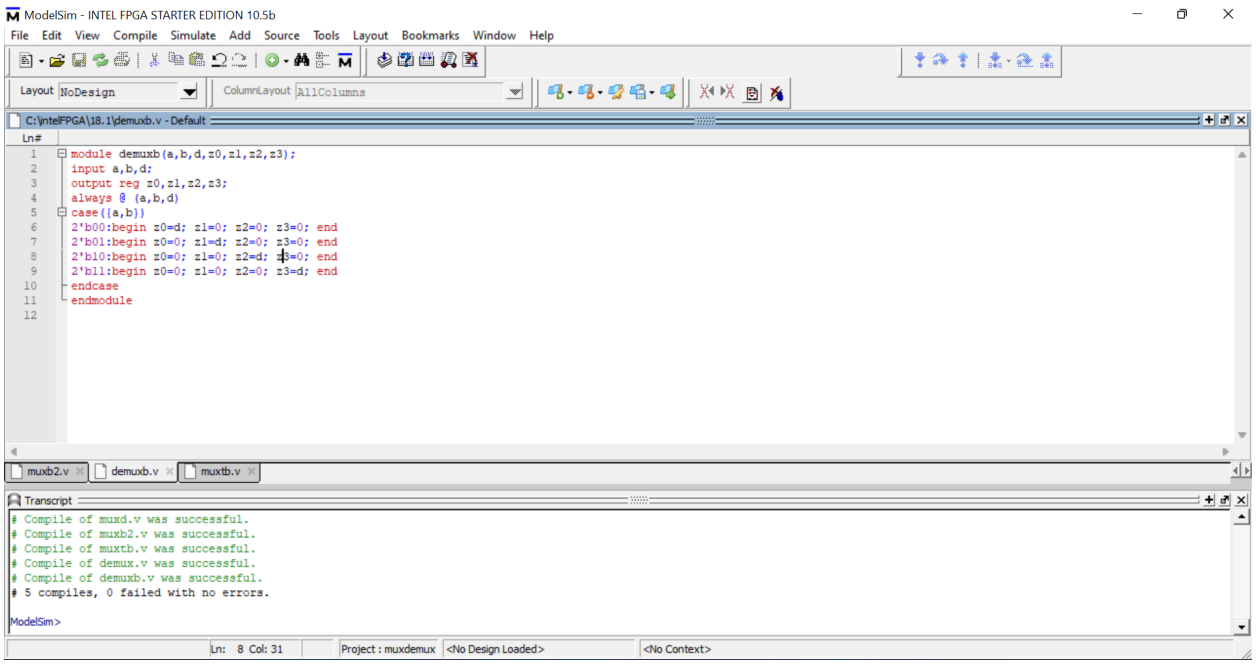
```
module demux(d,a,b,z0,z1,z2,z3);
input d,a,b;
output z0,z1,z2,z3;
assign z0=((~a)&(~b)&d);
assign z1=((~a)&b&d);
assign z2=(a&(~b)&d);
assign z3=(a&b&d);
endmodule
```



<u>Behavioral Modelling:</u>

```
module demuxb(a,b,d,z0,z1,z2,z3);
input a,b,d;
output reg z0,z1,z2,z3;
```

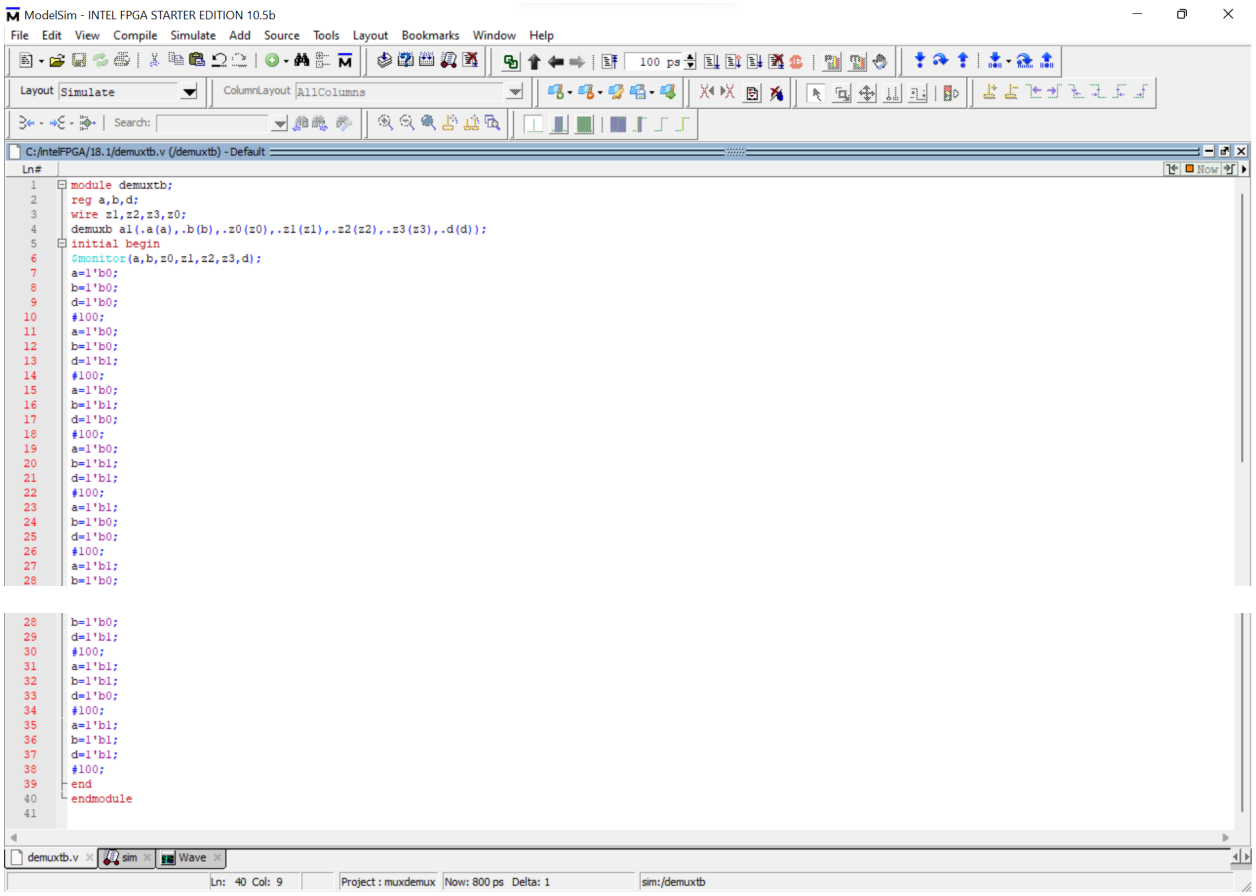
```
always @ (a,b,d)
case({a,b})
2'b00:begin z0=d; z1=0; z2=0; z3=0; end
2'b01:begin z0=0; z1=d; z2=0; z3=0; end
2'b10:begin z0=0; z1=0; z2=d; z3=0; end
2'b11:begin z0=0; z1=0; z2=0; z3=d; end
endcase
endmodule
```



<u>Test Bench:</u>

```
module demuxtb;
reg a,b,d;
wire z1,z2,z3,z0;
demuxb a1(.a(a),.b(b),.z0(z0),.z1(z1),.z2(z2),.z3(z3),.d(d));
initial begin
$monitor(a,b,z0,z1,z2,z3,d);
a=1'b0;
b=1'b0;
d=1'b0;
#100;
a=1'b0;
```

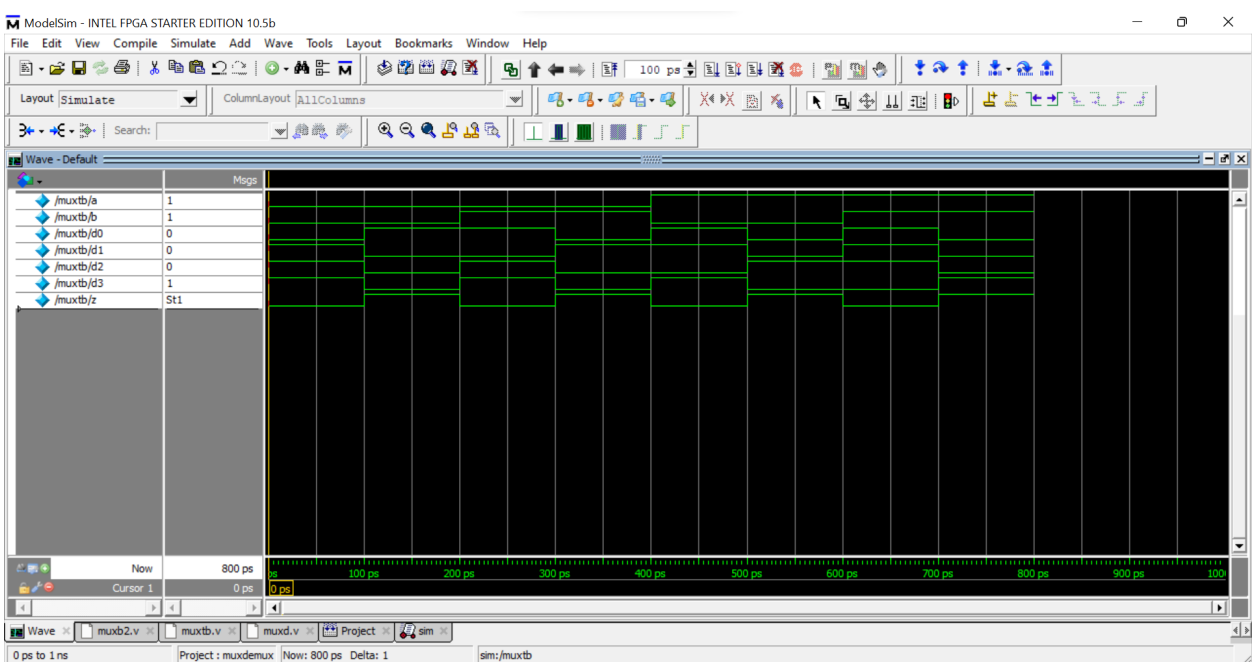
```
b=1'b0;
d=1'b1;
#100;
a=1'b0;
b=1'b1;
d=1'b0;
#100;
a=1'b0;
b=1'b1;
d=1'b1;
#100;
a=1'b1;
b=1'b0;
d=1'b0;
#100;
a=1'b1;
b=1'b0;
d=1'b1;
#100;
a=1'b1;
b=1'b1;
d=1'b0;
#100;
a=1'b1;
b=1'b1;
d=1'b1;
#100;
end
endmodule
```



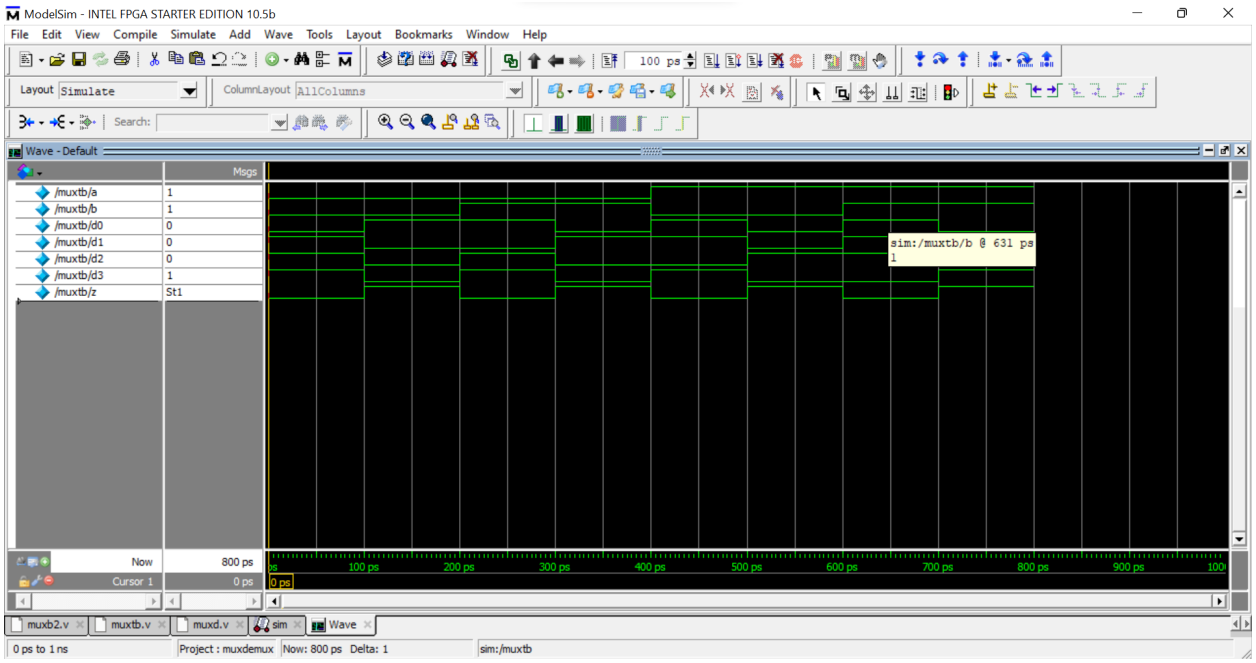
Output:

Multiplexer:

Data-Flow Modeling:

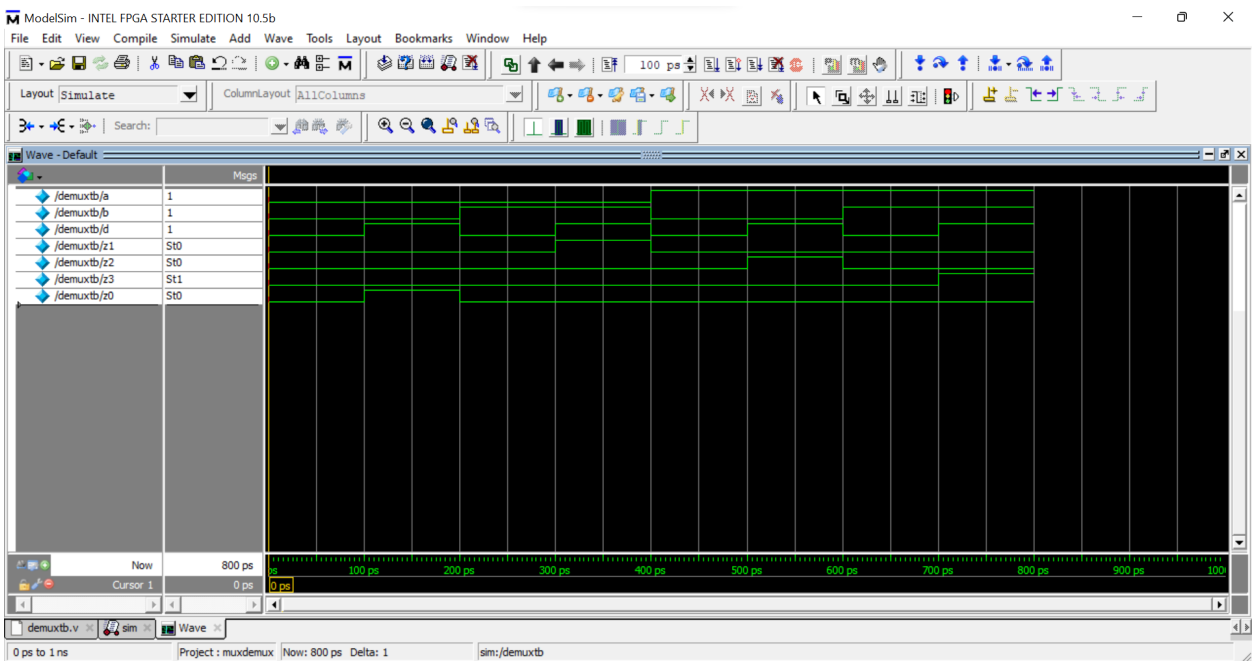


Behavioral Modeling:

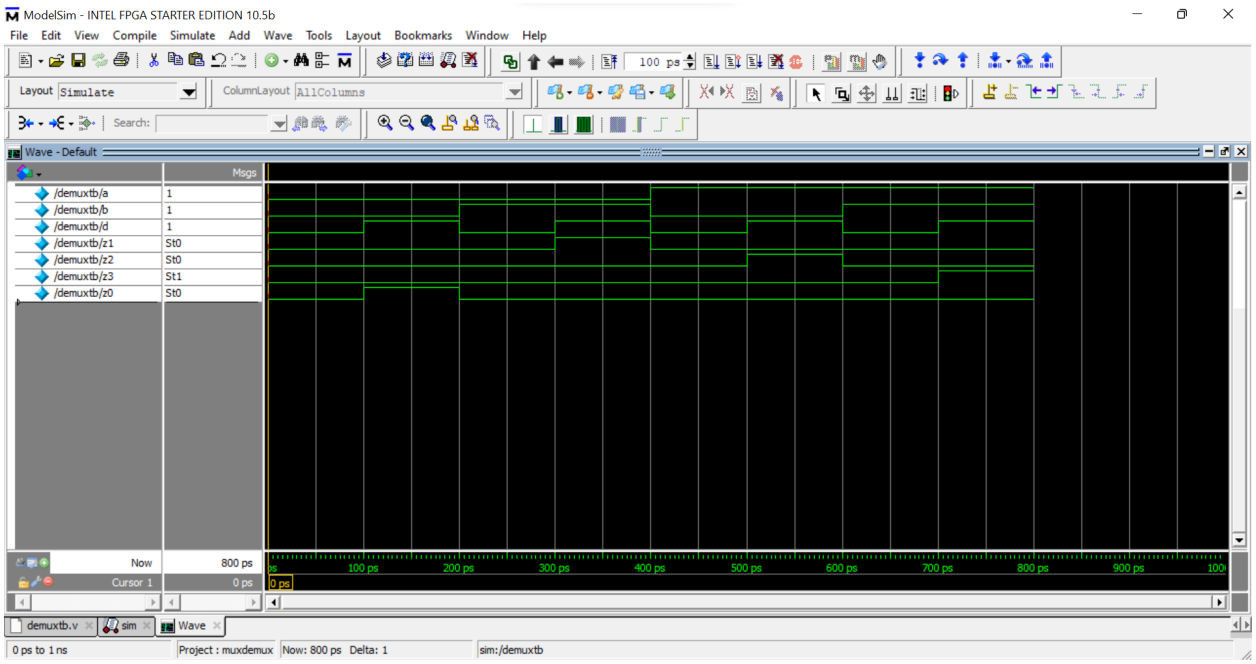


Demultiplexer:

Data-Flow Modeling:



Behavioral Modeling:



Result/ Inference:

From this experiment, the multiplexer and demultiplexer combinational circuits are verified using data-flow modeling and behavioral modeling using verilog programming.